

MarkIt Documentation (Plichtenheft)

- [MarkIt Documentation \(Plichtenheft\)](#)
 - [Requirements and Versions](#)
 - [Implementation description](#)
 - [Problems and Solutions](#)
 - [Supabase](#)
 - [Tree Views in WPF](#)
 - [General Bugs](#)
 - [Architecture and Functions](#)
 - [Tests](#)
 - [Instruction Manual](#)
 - [Timetable](#)
 - [Licenses](#)

Requirements and Versions

MarkIt only runs on Windows. You **do not** need to install further requirements. Please be sure, you have a stable internet connection. If not there might be some issues with export to pdf.

Implementation description

The development of our Markdown Editor started with the **planning phase**. In this phase, we discussed the features we wanted to implement, created a basic project structure, and decided which technologies we would use.

After the planning was finished, we started implementing the **login and registration system**. This allowed users to create accounts and securely log into the application.

User data and authentication were handled through **Supabase**.

Once the authentication system was working, we focused on the **user interface**. We designed the editor layout, created menus and settings pages, and improved the overall appearance of the application to make it more user-friendly and visually looking good.

The next step was implementing the **main editor functions**. We added the ability to write Markdown text as well as save and load documents. This allowed users to store their work and continue editing it later.

After the basic editor features were completed, we implemented **live rendering for Markdown elements**. This feature displays the formatted output in real time while the user is typing, making the editing process more convenient and interactive.

For the backend infrastructure, we used **Supabase**. At the beginning of the project, the Supabase server was hosted on a server located at Max grandparents' house in Berlin because port forwarding was available there. However, this setup was not very reliable because the server was not always running and sometimes caused connection problems.

To improve reliability, we moved the server to our local environment. We then used **Cloudflare Tunnels** to make the server reachable from the [www](https://www.cloudflare.com). Since we used the free version of Cloudflare Tunnels, the generated domain changed regularly.

To solve this issue, we created a system that automatically updated the current tunnel domain in a **GitHub Gist**. The client application then fetches the latest domain through the GitHub Gist API whenever it needed to connect to the server. This made sure that users could always reach the backend, even when the tunnel address changed.

After all **must-have features** were implemented, we focused on:

- Fixing bugs
- Improving stability
- Updating the documentation

Once everything was tested and reviewed, the project was prepared for the final submission.

Problems and Solutions

Supabase

As mentioned above, we encountered the problem with the reliability of the server at Max grandparents' server. Because many new and big applications like **ChatGPT**, **Discord**, **Stack Overflow** oder **Siemens**, we bumped into **Cloudflare**. We searched it up and could incorporate it into our server system. Through **GitHub Gist**, the user could access the dynamic URL whenever he wants to reach the server.

Tree Views in WPF

In addition, the **Tree Views** in the file workspace took a big amount of time. The only difficult part was checking **which item the user has selected**.

Why was it so hard?

Because we created a different Tree View Item for each file and folder. If you click on an item, it gets selected. However, if you click anywhere else to uncheck this item, it is still selected in the code behind. Also, you can check many different items at the same time, which was not intended. So we had to fix both issues, but it was very time consuming.

General Bugs

We also had some general and small problems like all other programming projects. However, the **two problems above** were the most time-consuming and difficult problems to fix.

Architecture and Functions

Every programm start works the same. First the code checks if it reaches the server. If not, you can only use the local functions, like writing, saving, formatting. But if the server is reachable, you can also use the cloud features like cloud sync. And general our programm works like all other text editors.

Tests

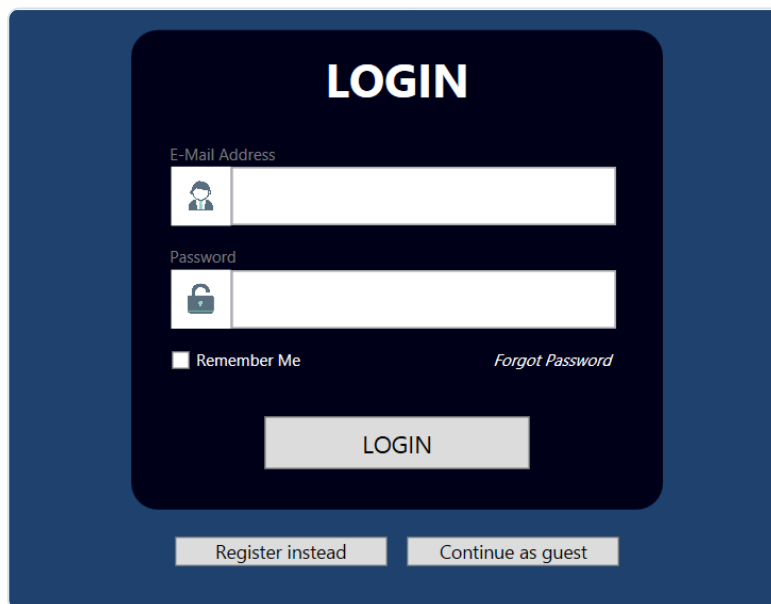
To test our application, we performed **manual testing** throughout the development process. As in every project, we worked on **feature branches**:

1. Karim develops a feature.
2. Karim tests the feature before creating a pull request.
3. Max reviews the code and tries out the feature.
4. The pull request gets merged into the main branch.

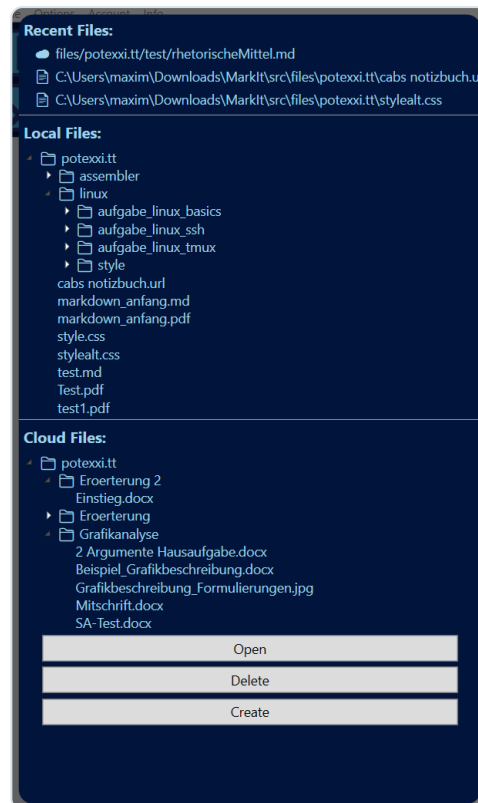
So every feature and its code were checked **twice** before being merged.

In the end, we tested **all features together** and after these tests, we published the application.

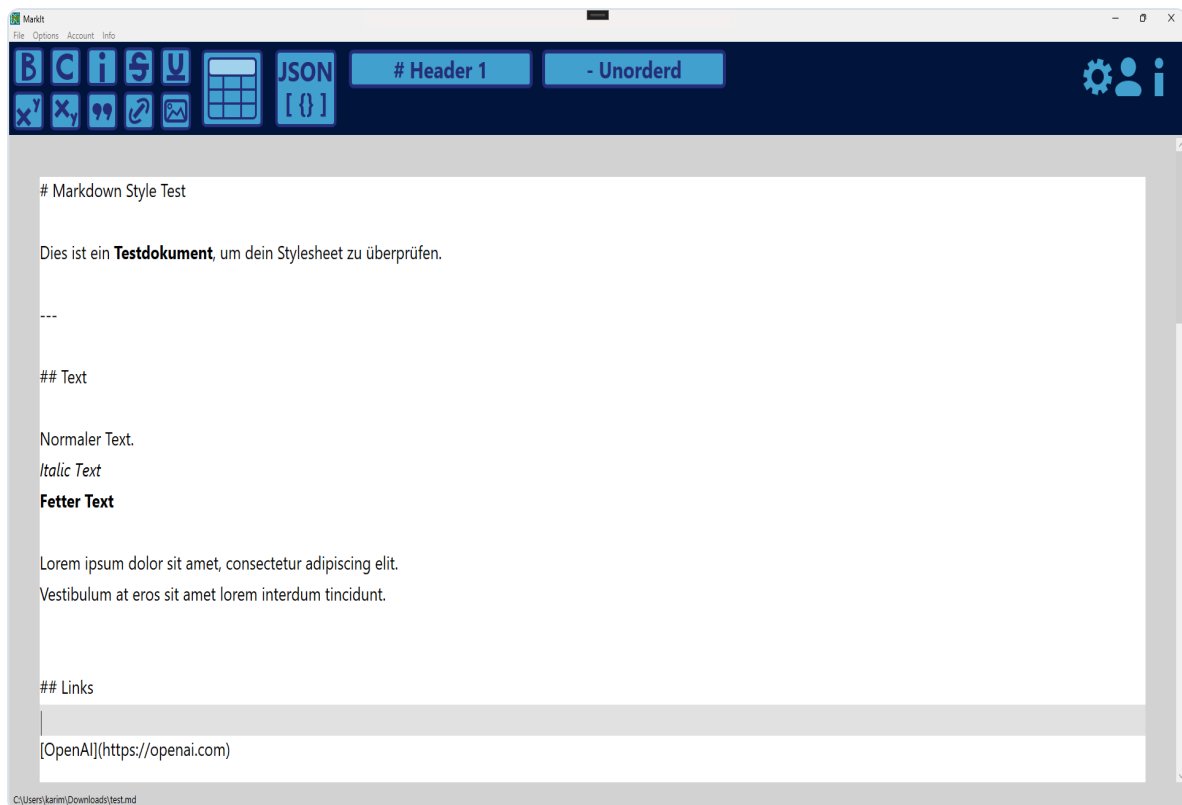
Instruction Manual

A screenshot of a login form titled "LOGIN" in white text on a dark blue background. The form is centered and contains two input fields: "E-Mail Address" with a user icon and "Password" with a lock icon. Below the password field is a checkbox labeled "Remember Me" and a link labeled "Forgot Password". A large "LOGIN" button is positioned below these elements. At the bottom of the form, there are two buttons: "Register instead" and "Continue as guest".

If your first start our programm you land at our login/registration window. Here you have to login/register. The registration works very easily. Enter an email, a secure password and you will recieve a verification code. Enter this code and you are good to go. If you forget your password, just click on *forget password* and you will recieve a email, in which you can change your password.



After you logged in successfully or continued as guest you will land at our workspace. Here you can open/create/delete or rename your files. If you log in, you can use the cloud sync and save files into our self hosted server system. With that you can manage your files from all other devices and locations.



After you selected a file you land at our main worksheet. If you click any button you will get into your worksheet the right content for your operation. In the right top corner you see the icons for the different subwindows: Settings, Profil Settings and Infos. There you can configure and costumize your local application.

These are all main features and functions. If we would describe all features we made (export to pdf, creating files, renaming files, changing colors...) it would take at least 20 pages of dinA4. So discover **MarkIt** at your own and have fun trying or using MarkIt!

Timetable

You can view our timetable (*when did someone something*) in the extra file called **time_table.md**.

Licenses

You can view our licenses of all graphics in the extra file called **licenses**.